

# Linux and Mac File Systems

## 1. Introduction to Ext4 and File Systems in Linux

The Ext4 (Fourth Extended File System) is a high-performance journaling file system used in Linux systems, succeeding Ext3. It supports extremely large storage capacities, offering a maximum individual file size of 16 TB and overall file system sizes up to 1 EB (exabyte). Ext4 introduces features like delayed allocation, journal checksums, faster file system checks, and improved timestamps. It also supports extent-based file storage, which reduces fragmentation and improves performance. Ext4 is widely adopted in most Linux distributions due to its balance of reliability, performance, and compatibility with older file systems like Ext2 and Ext3.

---

## 2. Everything is a File in UNIX/Linux

In the UNIX/Linux operating system, a fundamental concept is that “everything is a file.” This abstraction treats hardware devices, sockets, named pipes, and even processes similarly to regular files. It allows for a unified interface for interacting with different system components. For example, `/dev/sda` represents a hard drive, and `/proc/cpuinfo` provides CPU details—all accessed like files. This design simplifies tools and utilities, as a single file-oriented interface can manage a wide range of resources. It also aids in scripting and automation, enabling developers and administrators to handle I/O in a consistent and predictable manner across different objects.

---

## 3. File System Structure in Ext4

The Ext4 file system is composed of several distinct block types that organize and manage data storage efficiently. These include the **boot block**, **superblock**, **inode table**, and **data blocks**. The file system is divided into block groups for scalability and quick access. Each group contains a replica of the superblock, an inode bitmap, a block bitmap, and the inodes and data blocks themselves. By structuring storage in this modular way, Ext4 ensures fast lookups, reliable journaling, and space-efficient storage. Understanding each block’s role is critical for tasks like forensic recovery, system repair, and low-level disk operations.

---

### a) Boot Block

The boot block is the very first section of a storage device and typically occupies the first 512 bytes. It contains the bootloader or bootstrap code responsible for initializing the system and loading the operating system kernel into memory during startup. Although not part of the Ext4 file system itself, it's located just before the file system begins. This region is essential for system booting. It's usually written during system installation. Corruption in the boot block can make the system unbootable, requiring tools like GRUB or other boot managers to restore or reconfigure it for successful system loading.

---

### b) Superblock

The superblock is one of the most critical structures in the Ext4 file system. It stores metadata about the entire file system such as total number of inodes and blocks, block size, file system state, UUID, mount status, and supported features. Without the superblock, the file system cannot be mounted or read correctly. To safeguard against corruption, Ext4 stores multiple backup copies of the superblock across different block groups. This redundancy ensures that recovery tools can reconstruct the file system if the primary superblock becomes damaged, making the superblock a foundational component for integrity and stability.

---

### c) Inode Block

Inodes are data structures that store all metadata related to a file or directory, except for its name and path. Each inode holds information such as file size, permissions, ownership, timestamps (modified, accessed, changed), and pointers to data blocks. Inodes are stored in the inode block, with a fixed number created during file system formatting. Every file has a unique inode number. The system uses this number to locate the file's actual data on the disk. Because inodes do not store filenames, directories function as name-to-inode mapping tables. Efficient inode management is key to fast file access and system performance.

---

### d) Data Block

Data blocks are the basic units where actual file content is stored. After metadata is resolved via an inode, the system uses the inode's block pointers to retrieve or write file data into these blocks. The size of each data block is typically 1024, 2048, or 4096 bytes, depending on how the file system was created. Data blocks can be directly linked through the inode or indirectly accessed via multi-level pointers. In Linux forensics or data recovery, analyzing data blocks

directly allows recovery of deleted or corrupted files. Efficient allocation and deallocation of data blocks affect overall system performance and fragmentation.

---

#### 4. Understanding Inodes in Ext4

An inode is the fundamental metadata unit for each file in the Ext4 system. When a new file is created, an inode is assigned to store its attributes such as ownership, access permissions, timestamps, and pointers to data blocks. Unlike Windows systems, Linux does not store filenames in the inode; instead, directories act as lookup tables that map filenames to inode numbers. This separation allows efficient hard link implementation where multiple names can refer to a single file's inode. Inodes enable robust file operations, and understanding them is crucial for performing tasks like recovery, auditing, and permission management in Linux systems.

---

#### 5. Pointer System in Inodes

Inodes in Ext4 contain 13 block pointers used to locate the file's data. The first 12 pointers are structured to handle both small and large files efficiently. The first 10 are **direct pointers** that directly reference data blocks. The 11th pointer is **single indirect**, pointing to a block containing more block addresses. The 12th is **double indirect**, referring to a block of blocks of pointers. The 13th is **triple indirect**, allowing massive file sizes by extending the pointer chain further. This hierarchical pointer system balances performance and scalability, ensuring both small and huge files can be managed efficiently.

---

#### 6. Special Inodes and Hidden Data

Ext4 reserves several inodes for special functions. Inode 1 is the **bad block inode**, which points to sectors identified as damaged or unusable. Attackers sometimes exploit this to **hide malicious data**, since many forensic tools and operating systems avoid these blocks during scans. Inode 2 always refers to the root directory (/). Other system-reserved inodes handle journaling, recovery, and quota tracking. Understanding these special inodes is essential in forensics and low-level disk analysis, especially when investigating malware persistence techniques, hidden backdoors, or recovering corrupted systems where critical metadata might have been tampered with or bypassed by attackers.

---

## 7. Hard Links and Symbolic Links

A **hard link** is an additional filename pointing to the same inode, meaning it shares the same metadata and content as the original file. Deleting one hard link does not remove the file's content until all links are removed. In contrast, a **symbolic link (symlink)** is a separate file that points to the **path** of another file. Symlinks have their own inode and can link across different file systems or even remote paths. While hard links are efficient and reliable, symlinks offer flexibility. Understanding both is vital for managing storage, avoiding data duplication, and ensuring file reference integrity.

---

## 8. Directory Structure and Links

In Ext4, a directory is simply a file that maps filenames to inode numbers. Each directory contains entries for **.** (itself) and **..** (its parent), forming a structured tree that the operating system uses for path resolution. The link count of a directory reflects the number of subdirectories it contains, as each subdirectory contributes a reference via its **..** link. Understanding this is crucial in forensics, as it helps determine how many directories link to a given inode. It also explains why a directory's hard link count is never less than two and increases with every subdirectory creation inside it.

### Overview of macOS File Structures

#### 1. Dual-Fork File Design (Older macOS Versions):

- **Data Fork:** Stores user-generated data (e.g., text, spreadsheets).
- **Resource Fork:** Stores metadata and app-related elements like:
  - Resource maps
  - Header info
  - Icons
  - Window locations
  - Menus, dialog boxes, executable code
- Either fork can be empty depending on the file type.

#### 2. Volume Structure:

- A **volume** is a storage unit (entire disk or partition).
- **Logical block:** Basic unit of data (≤512 bytes).
- **Allocation block:** A group of logical blocks used for storing files.
  - A file always uses at least **one allocation block**, even if it only stores a few bytes (e.g., 11 bytes uses 512 bytes).

#### 3. EOF Concepts in HFS/HFS+:

- **Logical EOF:** Actual end of the file's data (e.g., byte 510).

- **Physical EOF:** Total space allocated on disk for the file (e.g., byte 1023).
- 4. **Fragmentation & Clumps:**
  - **Clumps** = Groups of contiguous allocation blocks.
  - Used to minimize file fragmentation as file sizes grow.
  - Helps maintain volume performance by grouping large files.

## macOS Forensics Procedures – Quick Summary

### 1. File Structure Differences:

- Linux: /home/username, /root
- macOS: /Users/username, /private/var/root
- /home exists but is empty on macOS.

### 2. File Access & Accounts:

- Limited access to other users' data.
- Guest account disabled by default; if enabled, no password and files are deleted on logout.

### 3. File Types & Storage:

- Settings stored in:
  - Plaintext
  - plist files (XML or binary)
  - SQLite databases
- Plist viewer tools: PlistEdit Pro, Apple Developer Tools
- SQLite viewer: SQLite Database Browser

### 4. Logging & Diagnostics:

- Logs stored in: /var/db/diagnostics, /var/db/uuid.text
- Use log, log collect, log show commands for unified logging.

### 5. Important Forensics Files:

- OS version: /System/Library/CoreServices/SystemVersion.plist

- Network interfaces:  
/Library/Preferences/SystemConfiguration/NetworkInterfaces.plist
- DHCP leases: /private/var/db/dhcpclient/leases
- User data:
  - Pre-10.7: /private/var/db/DirectoryService/flatfile.db
  - Post-10.7: /private/var/db/dslocal/nodes/Default/users
- Password hashes: /private/var/db/shadow/hash

#### 6. Encryption:

- FileVault (v10.3): Encrypted /Users directory; vulnerable to RAM attacks.
- FileVault2: Full disk encryption with AES-128, more secure.
- Passwords stored in keychains: /System/Library/Keychains, /Library/Keychains

#### 7. Deleted Files:

- Normal deletion: Moved to Trash (retains filename).
- CLI deletion: Bypasses Trash.
- macOS preserves file creation timestamps (unlike Windows).

#### 8. Forensics Tools:

- **Commercial:** BlackBag, SubRosaSoft, EnCase, X-Ways, FTK, Recon
- **Free:** Sleuth Kit, Autopsy
- Tools like MacQuisition create forensic images and manage disk arbitration.

#### 9. Disk Imaging Challenges:

- Mac Mini/MacBook: Difficult to open without special tools.
- Need a macOS-compatible forensic boot CD.
- Image is stored on external media (USB/FireWire).

#### 10. Tool Usage:

- Autopsy + Sleuth Kit: Used for examining images.
- Foremost: File carving tool with customizable config.

- Autopsy setup in Linux/macOS requires sudo, and Sleuth Kit must be installed first.